ML xYARDS NFL Expected Yards Predictor

Thomas Årland 30/10/2026

I built ML xYARDS to explore a simple question: given the game situation, how many yards would an NFL offense be expected to gain on its next play? The website compares a run with a pass or dropback and shows both estimates on a football field. The final model performs slightly better than a simple mean baseline, but its test R^2 is only 0.033. I therefore present the app as an educational tool, not as a play-calling recommendation.

1 DESCRIBE THE PROBLEM

SCOPE

The model predicts recorded yards gained from information available before the snap: play choice, down, yards to go, field position, score difference, quarter, and time remaining. A fan or student can change these inputs and immediately see how the expected gain changes. This makes it easier to explore patterns that would be slow to calculate by repeatedly filtering raw play-by-play data.

The product is meant for learning and exploration. Public statistics sites already show play results and team summaries, and a manual analysis could average similar historical plays. Machine learning is useful here because it can combine several game-state variables and return a consistent estimate even when an exact situation has few close matches.

The full pipeline starts with nflverse data, applies the same cleaning and feature rules to every season, trains several candidate models, and saves the selected scikit-learn pipeline. The Streamlit website loads the saved model and the evaluation files. If the cleaning rules or features change, the model, metrics, tests, and app must all be updated together. The project runs on a normal computer with Python 3.12 and open-source libraries. A fresh download needs several hundred megabytes of network traffic and roughly 2-4 GB of free memory, but the website itself only loads the saved artifacts.

METRICS

I used validation RMSE to select the model because the project estimates a mean outcome and the models are trained with squared error. I also report MAE, which is easier to read as an average miss in yards, and R^2, which compares the model with a constant prediction. The minimum technical goal was to beat the training-mean baseline on 2024 data and deliver a working, reproducible website.

Measure	What it tells me	Result
2024 validation RMSE	Main model-selection measure	8.441 yd
Baseline validation RMSE	Reference for a constant prediction	8.594 yd

Measure	What it tells me	Result
2025 test MAE	Average absolute miss on unseen plays	5.518 yd
2025 test R^2	Variation explained beyond a constant	0.0329

The model clears the technical threshold: its validation RMSE is 1.78% lower than the baseline. That is a real but modest improvement. Because this is a non-commercial student project, I did not set a profit target. The user-facing goals are simpler: the app should respond correctly, explain what the prediction means, show the evidence behind it, and be reproducible from the public repository.

2 DATA

SOURCE AND SPLIT

The data comes from the nflverse play-by-play collection and is loaded with nflreadpy. I used regular-season and postseason games from 2021 to 2025. The source files contain 247,284 rows. After selecting relevant plays and removing missing, invalid, and overtime records, 174,581 plays remain: 105,928 pass or dropback plays and 68,653 runs.

Split	Seasons	Plays	Use
Training	2021-2023	105,580	Fit preprocessing and models
Validation	2024	34,728	Choose the model
Test	2025	34,273	Final evaluation

The chronological split is important. A random split would mix plays from the same games and seasons across training and evaluation. Using later seasons gives a more realistic check of how the model behaves on future data. The selected model is not refitted on 2024, so the validation and test results describe the same fitted pipeline.

FEATURES LABELS AND CLEANING

The target is recorded yards gained, including losses and zero-yard plays. The model only receives information that could be known before the snap. It does not receive completion status, air yards, touchdowns, the post-play score, or the target itself. Game and play identifiers are kept for auditing and splitting, but they are not model features.

The cleaning step removes duplicates, special-teams plays, no-play events, kneels, spikes, two-point attempts, overtime, invalid clocks or downs, missing model values, and physically impossible distances or gains. Scrambles are grouped with passes and dropbacks to better reflect the intended call, and sacks remain in that group. This is still an approximation because some options and broken plays do not have a clear pre-snap category.

The data describes public professional games and does not contain private personal records. The larger ethical issue is interpretation. Historical play choice is affected by team strength,

opponents, personnel, and coaching strategy. Those factors are only partly represented, so changing the play-choice input does not reveal the causal effect of calling a run instead of a pass.

3 MODELING

I compared four candidates: a mean baseline, Ridge regression, and histogram gradient boosting with either 15 or 31 leaves. Every candidate used the same preprocessing. Play choice was one-hot encoded, and the numerical features were standardized inside a pipeline fitted only on the training seasons. The comparison was deliberately small. I selected the winner with 2024 validation RMSE and used the 2025 test set only once for final evaluation.

Candidate	MAE	RMSE	R^2
Mean baseline	5.768	8.594	-0.0001
Ridge	5.613	8.481	0.0260
Boosting 15 leaves	5.578	8.441	0.0351
Boosting 31 leaves	5.586	8.452	0.0327

The 15-leaf boosting model won. The 31-leaf version fitted the training data slightly better but performed slightly worse on validation data. This is a useful reminder that a more flexible model is not automatically better on new plays.

2025 test result	Overall	Run	Pass or dropback
MAE yards	5.518	3.590	6.790
RMSE yards	8.279	6.102	9.445
R^2	0.0329	0.0240	0.0198

Errors are larger for passes and dropbacks because their outcomes range from sacks and incompletions to long gains. The low R^2 shows that the available game-state variables explain little of what happens on one specific play. A prediction near the average can still be reasonable as an expected value, but it should not be read as the most likely result.

Permutation importance gave the clearest interpretation of what the model uses. On 6,000 validation plays, shuffling field position increased RMSE by about 0.176 yards and shuffling play choice increased it by about 0.133 yards. Every other feature changed RMSE by less than 0.010 yards on average. These are measures of model reliance, not causal effects.

4 DEPLOYMENT

The selected joblib pipeline is loaded by a Streamlit app with three tabs. The Play explorer validates the inputs and updates the predictions and field automatically. The Model analysis tab shows the measured validation and test results, distributions, residuals, error by down, and feature importance. The Method and sources tab explains the temporal split, data source, and limits of the comparison.

Repository: github.com/ThomasAarland/MLxYARDS

Website: https://mlxyards.streamlit.app

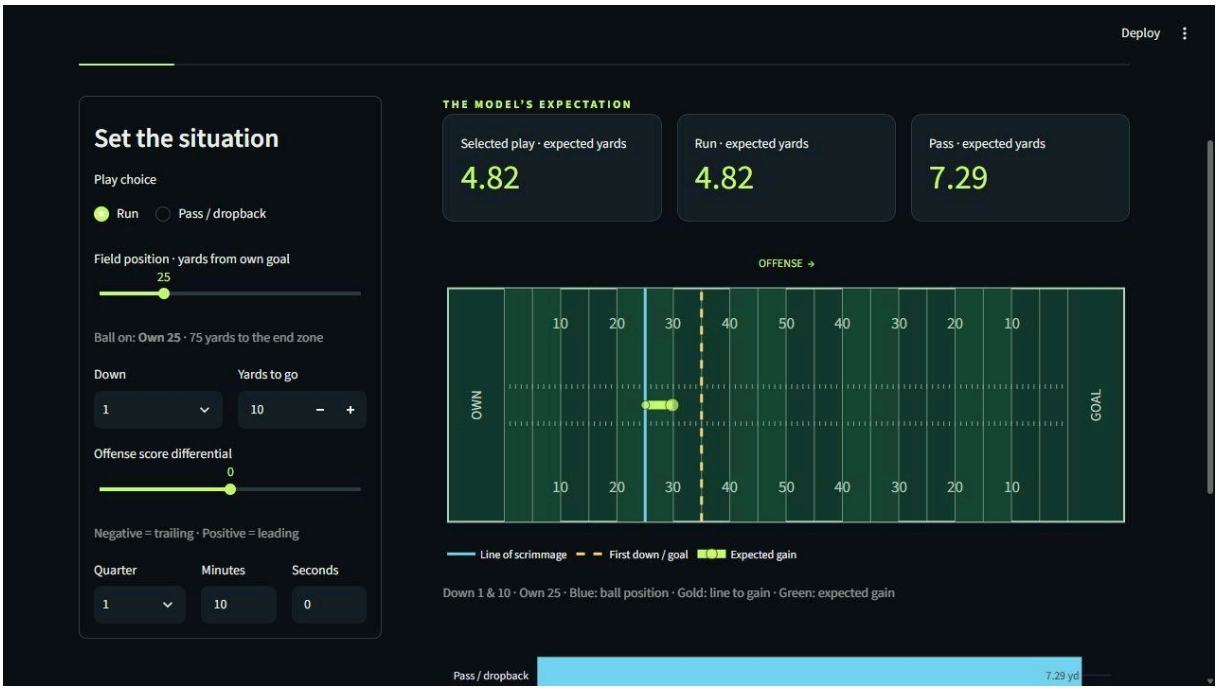


Figure 1 The Streamlit play explorer using the saved model

MONITORING AND NEXT STEPS

A deployed version should track input errors, model-loading failures, response time, and changes in the input distribution. When a new season is complete, the frozen model should be evaluated before it is retrained. A clear rise in MAE or RMSE, or a systematic residual pattern by play type or down, would be a reason to investigate.

The next useful features would be team strength calculated without future information, personnel, formation, and weather. A quantile model could also show a range of plausible outcomes instead of only the mean. For tactical use, expected points or win probability would be more relevant than yards alone.

AI ASSISTANCE

OpenAI Codex assisted with the implementation, experiment setup, test execution, interface design and documentation. I remain responsible for report writing and reviewing the submitted work and understanding the model, code, results, and limitations.

5 REFERENCES

Sources were accessed on 26 October 2026 unless another date is shown.

[1] nflverse data repository and play-by-play releases. Historical records used for training and evaluation. CC BY 4.0. https://github.com/nflverse/nflverse-data
[2] nflreadpy documentation. Data-loading interface used by the project. https://nflreadpy.nflverse.com/
[3] nflreadr play-by-play dictionary. Definitions for field position, clocks, score margin, yardage, and play categories. https://nflreadr.nflverse.com/articles/dictionary_pbp.html

[4] scikit-learn documentation. Model, pipeline, and permutation-importance references. https://scikit-learn.org/stable/
[5] Streamlit documentation. Web app, testing, and Community Cloud guidance. https://docs.streamlit.io/
[6] Plotly Python documentation. Interactive field and analysis graphics. https://plotly.com/python/
[7] HVL DAT158 assignment 2 brief. Course requirements and lifecycle structure supplied with the assignment template.
[8] OpenAI Codex. AI-assisted work session used for implementation, testing, documentation, and report drafting on 26-30 October 2026.